

Hoja de Ruta — De una PC vacía a la aplicación publicada (v22)

Curso de Desarrollo Web Asistido con IA

Leyenda de frecuencia: - ● Se hace una sola vez (para siempre, no se repite) - ● Se hace por cada proyecto nuevo (cada aplicación distinta que armes) - ● Se hace en cada sesión de trabajo (cada vez que te sentás a programar)

Stack de esta capacitación: **OpenCode** (agente de IA) + **Next.js** (JavaScript + Tailwind) + **Supabase** (base de datos, conectado por MCP) + **Vercel** (publicación).

PARTE 1 — Instalación y Configuración del Entorno

(Todo lo de esta parte se hace una sola vez, en cada PC)

1. ● Crear cuenta en GitHub 1.1) Ir a <https://github.com/join> 1.2) Completar correo, contraseña y nombre de usuario 1.3) Verificar el correo

GitHub es donde vamos a guardar el historial de todo lo que construyamos, como una caja fuerte con copias de cada versión de tu proyecto.

2. ● Instalar VS Code 2.1) Ir a <https://code.visualstudio.com/download> 2.2) Descargar el instalador para Windows 2.3) Ejecutarlo, aceptar todo con “Next” y “Finish”

Es el programa donde vamos a ver y editar el proyecto. Tiene una terminal integrada, que es la ventana negra donde le vamos a hablar a la IA.

3. ● Instalar Node.js 3.1) Ir a <https://nodejs.org> 3.2) Descargar la versión **LTS** (la recomendada, más estable) 3.3) Ejecutar el instalador, “Next” hasta el final 3.4) Verificar: abrir la terminal y escribir `node -v` (debe mostrar un número de versión)

Node.js es el motor que necesitan todas las herramientas que vamos a usar (OpenCode, Next.js). Sin esto no arranca nada.

4. ● Instalar Git 4.1) Ir a <https://git-scm.com/downloads> 4.2) Descargar el instalador para Windows 4.3) Ejecutar, dejar todas las opciones por defecto, “Next” hasta el final

Es el programa que le permite a tu PC hablar con GitHub.

5. ● Configurar Git en la PC, vinculado a GitHub 5.1) Abrir la terminal (desde VS Code: menú Terminal — New Terminal) 5.2) Escribir: `git config --global user.name "Nombre Apellido"` 5.3) Escribir: `git config --global user.email "correo@ejemplo.com"` (el mismo correo de GitHub) 5.4) La primera vez que hagas un `git push` (más adelante) se va a abrir una ventana del navegador pidiendo iniciar sesión en GitHub 5.5) Iniciar sesión ahí y autorizar

6. ● Instalar OpenCode 6.1) Abrir la terminal 6.2) Escribir: `npm install -g opencode-ai` 6.3) Verificar: escribir `opencode --version`

OpenCode es el agente de IA que va a escribir el código por vos, a partir de lo que le pidas en lenguaje simple.

7. ● Configurar el modelo de IA en OpenCode 7.1) Escribir en la terminal: `opencode` 7.2) Adentro de OpenCode, escribir: `/models` 7.3) Elegir **DeepSeek V4 Flash Free** (vía OpenCode Zen)

⚠ Usamos DeepSeek V4 Flash Free porque no requiere un login externo complicado — otras opciones (como GitHub Copilot dentro de OpenCode) pueden entrar en loops de autenticación que traban la clase.

8. ● Crear cuenta en Supabase 8.1) Ir a <https://supabase.com> 8.2) Click en “Start your project” o “Sign in” 8.3) Elegir “Continue with GitHub”

Supabase es donde va a vivir la base de datos de tus aplicaciones (los datos que carguen los usuarios).

⚠ Usá esta cuenta de Supabase únicamente para practicar en este curso. Nunca uses esta misma cuenta para un proyecto real con datos de un cliente.

9. ● Crear cuenta en Vercel 9.1) Ir a <https://vercel.com> 9.2) Click en “Sign Up” 9.3) Elegir “Continue with GitHub”

Vercel es el servicio donde vamos a publicar la aplicación para que cualquiera pueda entrar desde internet.

PARTE 2 — Construcción de Software

Por cada proyecto nuevo (una aplicación distinta)

10. ● Crear repositorio en GitHub 10.1) Ir a <https://github.com/new> 10.2) Ponerle nombre al repositorio 10.3) Dejarlo en “Private” o “Public” según prefieras 10.4) Click en “Create repository”

11. ● Crear la carpeta local del proyecto 11.1) Crear una carpeta nueva en tu PC para este proyecto 11.2) Abrir esa carpeta en VS Code (Archivo — Abrir Carpeta)

12. ● Pedirle a OpenCode que cree el proyecto Next.js 12.1) Abrir la terminal dentro de esa carpeta 12.2) Escribir `opencode` 12.3) Pedirle en lenguaje simple: “Creá un proyecto Next.js con JavaScript y Tailwind CSS en esta carpeta”

13. ● Crear el proyecto (= base de datos) en Supabase 13.1) Entrar a <https://supabase.com/dashboard> 13.2) Click en “New Project” 13.3) Ponerle nombre, elegir una contraseña para la base (guardarla, aunque no la vamos a usar para conectar la app) y elegir región 13.4) Esperar a que el proyecto termine de crearse (unos minutos)

Este paso genera datos propios de ESE proyecto: el **Project URL** y la **Publishable key (anon key)** — estos dos sí se usan para conectar la app.

14. ● Conectar OpenCode con este proyecto de Supabase por MCP 14.1) Pedirle a OpenCode: “Conectá este proyecto al MCP de Supabase, usando el `project_ref` de mi proyecto [nombre]” 14.2) OpenCode va a pedirte autenticarte contra Supabase (se abre el navegador) — iniciá sesión y autorizá 14.3) Confirmá que quede **acotado a este único proyecto** (project scoping), no a toda tu cuenta

⚠ Esta es la parte más importante del capítulo. En vez de copiar una clave secreta de tu cuenta completa a mano, OpenCode se conecta directo a Supabase con un permiso limitado a este proyecto puntual. Todo cambio de estructura de datos (tablas, columnas, relaciones) que pidas de acá en más queda registrado como una migración versionada — si algo sale mal, se puede rastrear y revertir.

15. ● Pedirle a OpenCode que diseñe las tablas de la base de datos 15.1) Explicarle en lenguaje simple qué necesita guardar tu aplicación (por ejemplo: “necesito una tabla de clientes con nombre, teléfono y email”) 15.2) OpenCode va a proponer la estructura antes de aplicarla — revisala 15.3) Confirmar para que la aplique (usa el MCP para crear la migración real)

16. ● Copiar el Project URL y la Publishable key desde Supabase 16.1) En el dashboard del proyecto, ir a Settings — API 16.2) Copiar el **Project URL** 16.3) Copiar la **Publishable key** (**anon key**) 16.4) Pedirle a OpenCode que los guarde en un archivo `.env.local`

⚠ El archivo `.env.local` nunca se sube a GitHub. Es información que se queda solo en tu PC.

17. ● Pedirle a OpenCode que conecte Next.js con la base de datos 17.1) Pedirle: “Conectá esta aplicación Next.js con Supabase usando las variables del `.env.local`”

18. ● Pedirle a OpenCode que arme las pantallas 18.1) Describir en lenguaje simple qué pantallas necesitás (por ejemplo: formulario para cargar un cliente, listado para verlos) 18.2) Revisar el resultado en el navegador (ver Parte 2, Cada sesión de trabajo)

19. ● Conectar el repositorio en Vercel y publicar por primera vez 19.1) Ir a <https://vercel.com/new> 19.2) Elegir “Import” sobre el repositorio de GitHub de este proyecto 19.3) En “Environment Variables”, importar el archivo `.env` en bloque (no variable por variable) 19.4) Click en “Deploy”

⚠ Como `.env.local` nunca se sube a GitHub, este paso es obligatorio: sin cargar las variables acá, la aplicación publicada no va a poder conectarse a la base de datos.

Cada sesión de trabajo (mismo proyecto, distintos días o clases)

20. ● Abrir la carpeta del proyecto en VS Code

21. ● Abrir la terminal y lanzar OpenCode 21.1) Escribir `opencode`

22. ● Levantar el modo desarrollo 22.1) Pedirle a OpenCode (o escribir directo): `npm run dev` 22.2) Abrir en el navegador la dirección que indique (normalmente `http://localhost:3000`)

23. ● Pedirle cambios a OpenCode 23.1) Describir en lenguaje simple qué querés agregar, corregir o cambiar 23.2) Si el cambio afecta la estructura de datos (por ejemplo, agregar un campo nuevo), OpenCode te va a mostrar el plan antes de aplicarlo vía MCP — confirmalo

24. ● Probar los cambios en el navegador

25. ● Pedirle a OpenCode que guarde los cambios en GitHub 25.1) Pedirle: “Guardá estos cambios en GitHub” (hace commit y push)

26. ● El despliegue se actualiza solo Si ya conectaste el proyecto a Vercel (paso 19), la aplicación publicada se actualiza automáticamente al hacer push. No hay que hacer nada más.

PARTE 3 — Publicación Web Final

Esta parte ya queda cubierta dentro de la Parte 2 (pasos 19 y 26): una vez conectado el repositorio a Vercel con las variables de entorno cargadas, cada push a GitHub actualiza la aplicación publicada automáticamente. No hace falta un proceso de publicación aparte para cada cambio.

Nota: para cambios estructurales de la base de datos más avanzados (relaciones entre varias tablas, integridad referencial), hay un documento separado que profundiza en esos conceptos.